The Science Behind PIRX

Calculation Stack, ML Components, and Rollout Status 2026-03-11

PIRX Engineering

The Science Behind PIRX

Abstract

PIRX (Performance Intelligence Rx) is a projection-first running performance system that maps training structure to race capability. This paper documents the implemented equations, constants, thresholds, and model orchestration behavior currently present in the repository. It also records ML rollout status across deterministic, KNN, LSTM, Optuna lifecycle, and injury-risk components, with explicit separation between active production path, rollout-gated path, and non-primary/planned paths.

Keywords: running performance modeling, projection engine, ACWR, event scaling, model rollout, wearable analytics

Figure 1. End-to-End Data Flow

```
Wearable Sync -> Cleaning -> Feature Engineering
-> Projection Recompute
      |             |                 |
|
      v             v                 v
v
  activities    valid runs       feature vector
projection_state

driver_state

|

v
                                        APIs
(/projection, /drivers, /readiness)

|

v

Frontend + Realtime + Chat
```

1. Implemented Calculation Stack

1.1 Data Cleaning and Validation

The cleaning stage admits only running-relevant activities and applies hard bounds to remove invalid sessions before feature generation.

Table 1. Cleaning thresholds and gates

Signal	Rule
Allowed activity types	`easy`, `threshold`, `interval`, `race`
Non-race minimums	`duration_seconds >= 180` and `distance_meters >= 1600`
Race minimums	`duration_seconds >= 60` and `distance_meters >= 400`
Pace lower bound	reject when `pace_sec_per_km < 223`
Pace upper bound	reject when `pace_sec_per_km > 900`
Relative pace outlier	reject when `pace_sec_per_km > 1.5 * runner_avg_pace`
Elevation quality gate	reject outdoor `distance_meters > 10000` and `elevation_gain_m == 0`

Table 1 should be interpreted as hard eligibility gates before downstream equations are computed.

1.2 Feature Engineering Equations

Feature engineering computes volume, intensity, efficiency, consistency, and physiology features over rolling windows.

The weighted temporal aggregation used across the pipeline is:

`weighted_feature_score = 0.45 * W_7d + 0.35 * W_8_21d + 0.20 * W_22_90d`

where:

- W_{7d} = rolling_distance_7d
- W_{8-21d} = rolling_distance_21d/3
- W_{22-90d} = rolling_distance_42d/6

Table 2. Feature equations

Domain	Equation
Volume	`rolling_distance_7d = sum(distance_meters last 7d)`
Volume	`rolling_distance_21d = sum(distance_meters last 21d)`
Volume	`rolling_distance_42d = sum(distance_meters last 42d)`
Volume	`rolling_distance_90d = sum(distance_meters last 90d)`
Session density	`sessions_per_week = count(activities last 7d)`
Long-run exposure	`long_run_count = count(distance_meters >= 15000 last 42d)`
Intensity zones	`zN_pct = zone_time_N / total_zone_time, N in {1..5}`
Threshold density	`threshold_density_min_week = (z4_seconds / 60) / 3`
Speed exposure	`speed_exposure_min_week = (z5_seconds / 60) / 3`
Economy	`matched_hr_band_pace = mean(pace where 140 <= avg_hr <= 155)`
HR drift	`hr_drift_sustained = mean((second_half_pace - first_half_pace)/first_half_pace)`
Pace decay	`late_session_pace_decay = mean((last_quarter_pace - first_half_pace)/first_half_pace)`
Consistency	`weekly_load_stddev = std(sum(distance_meters per week) over 6 weeks)`
Consistency	`block_variance = var(sum(distance_meters per 14d block) over 3 blocks)`
Consistency	`session_density_stability = std(session_count_per_week over 6 weeks)`

The EWMA ACWR implementation is:

- `acute_alpha = 2 / (acute_days + 1)`
- `chronic_alpha = 2 / (chronic_days + 1)`

- `EWMA_t = alpha * load_t + (1 - alpha) * EWMA_(t-1)`
- `ACWR = acute_load / chronic_load`

with windows:

- `acwr_4w = (7, 28)`
- `acwr_6w = (7, 42)`
- `acwr_8w = (7, 56)`

1.3 Baseline Estimation (5K Anchor)

PIRX baseline estimation is tiered and uses the first available signal in this order:

1. Race-result detection (+ Riegel conversion when non-5K race)
2. Sustained hard effort
3. Discounted p10 pace
4. Adjusted median pace
5. Default fallback `1500` seconds

Table 3. Baseline estimator thresholds

Parameter	Value
`MIN_PACE`	`223 sec/km`
`MAX_PACE`	`900 sec/km`
`MIN_DISTANCE`	`1600 m`
Race HR gate	`avg_hr / estimated_max_hr >= 0.83`
Sustained effort HR gate	`avg_hr / estimated_max_hr >= 0.85`
Tier 3 formula	`tier3 = p10_pace * 5 * 0.96`
Tier 4 formula	`tier4 = median_pace * 5 * 0.80`

KNN cold-start support is available when baseline defaults and usable history exists through the reference population path.

1.4 Projection Engine

Driver scoring and aggregation:

`ratio = value / baseline`, `feature_score = clip(50 * ratio, 0, 100)`

Inverse-direction features use:

`ratio_inverse = 2 - min(ratio, 2)`

Driver weights:

- aerobic_base: `0.30`
- threshold_density: `0.25`
- speed_exposure: `0.15`
- running_economy: `0.15`
- load_consistency: `0.15`

Projection equations:

- `weighted_sum = sum(driver_score_d * weight_d)`
- `max_improvement = baseline_time * 0.25`
- `improvement_factor = (weighted_sum - 50) / 50`
- `total_improvement_seconds = improvement_factor * max_improvement`
- `raw_projected = max(baseline_time - total_improvement_seconds, 60)`

Volatility dampening:

`projected = alpha * raw_projected + (1 - alpha) * previous_projected`, with `alpha in [0.3, 0.7]`.

Structural shift threshold:

`abs(delta_projected) >= 2.0 seconds`

Supported range:

`total_pct = 0.015 + min(volatility / projected, 0.05) + (1 - data_quality) * 0.02 + acwr_pct`

where `acwr_pct = 0.01` when `acwr_4w > 1.5` or `acwr_4w < 0.6`, else `0`.

`range_low = projected * (1 - total_pct)`, `range_high = projected * (1 + total_pct)`

Driver contribution decomposition is constrained to equal total improvement exactly (final-driver remainder assignment).

1.5 Event Scaling

Core Riegel relation:

`T2 = T1 * (D2 / D1)^k`

with defaults and bounds:

- default `k = 1.06`
- individualized `k` clamped to `[1.01, 1.15]`

Training-volume adjusted exponent:

`k = max(0.98, 1.06 - 0.005 * ((weekly_km - 40) / 10))`, then `k <= 1.15`

5K boundary phase adjustment:

- crossing to longer event: `k += 0.02`
- crossing to shorter event: `k -= 0.02`

Environmental penalty:

`multiplier = 1 + 0.0035 * degrees_outside(10, 17.5)`

1.6 Readiness and Risk

Readiness score:

`score = 0.30*acwr_balance + 0.25*fatigue_freshness + 0.20*training_recency + 0.15*physiological + 0.10*consistency`

clipped to `[0, 100]`.

Implemented readiness labels:

- `Peak`
- `Good`
- `Moderate`
- `Low`
- `Very Low`

Injury-risk signal is additive and does not mutate projection state. The random-forest path computes:

- `risk_probability in [0, 1]`
- `risk_band = low (<0.35), moderate (<0.60), high (>=0.60)`

1.7 Trajectory and Scheduled Degradation

Scenario transforms:

- maintain: volume `1.00`, intensity `1.00`, consistency `1.00`
- push: volume `1.05`, intensity `1.15`, consistency `0.95`
- ease_off: volume `0.80`, intensity `0.90`, consistency `1.10`

Inactivity structural decay:

- `>10 days`: widen range by 5 percent each side; confidence decrement `0.05` floor `0.1`
- `>21 days`: status `Declining`; confidence decrement `0.1` floor `0.1`

Figure 2 summarizes serving-path behavior when model selection and fallback logic interact with the projection engine.

Figure 2. Projection Serving and Fallback Path

```
ModelOrchestrator
    |
    +--> deterministic (default production path)
    |
    +--> lstm selected?
           |
           +--> rollout/flag pass ->
LSTMInferenceAdapter -> projection override
           |
           +--> artifact missing/error ->
deterministic fallback
    |
    +--> knn selected?
           |
           +--> currently feature-flag gated for
serving
```

---

2. ML Components and Current Status

This section reflects repository state verified against current code paths and migration contracts.

Table 4. ML component status map

Component	Status	Code touchpoints	Role
Deterministic projection engine	Implemented and active	`projection_engine.py`, `projection_service.py`	Primary production projection path
KNN cold-start baseline	Implemented and active (baseline fallback stage)	`reference_population.py`, `projection_service.py`	Improves default baseline for sparse history
LSTM inference adapter	Implemented but rollout-gated	`lstm_inference.py`, `model_orchestrator.py`, `projection_service.py`	Optional projection override when active model/artifact exists
Optuna lifecycle + promotion scaffold	Implemented but rollout-gated	`ml_tasks.py`, `013_ml_lifecycle.sql`	Tracks studies/trials and model promotion metadata
Injury risk random-forest	Implemented and active (readiness additive)	`injury_risk_model.py`, `readiness.py`	Readiness risk signal and assessment persistence
LMC module	Implemented, non-primary	`lmc.py`	Alternate distance-time modeling path
SHAP explainer heuristics	Implemented, non-primary	`shap_explainer.py`	Explanation support outside main serving decision

Table 4 is the canonical implementation-status map for external technical communication and should be kept synchronized with code changes.

2.1 Model Lifecycle Persistence

Lifecycle tables are additive and support training/tuning/serving provenance:

- `model_registry`
- `model_training_jobs`
- `optuna_studies`
- `optuna_trials`
- `model_artifacts`
- `injury_risk_assessments`

Projection metadata contracts include:

- `model_source`
- `model_confidence`
- `fallback_reason`

and `projection_state.model_type` includes:

- `deterministic`
- `lmc`
- `knn`
- `lstm`
- `gradient_boosting`

Figure 3. Model Lifecycle Entities

```
model_registry (model_family, status, metadata)
        |
        +--> model_training_jobs (train/tune job
runs)
                 |
                 +--> optuna_studies
                 |       |
                 |       +--> optuna_trials
                 |
                 +--> model_artifacts (weights,
metrics, config)

projection_state stores serving metadata:
  model_type, model_confidence, fallback_reason
```

3. Formula and Constant Quick Reference

The constants in Table 5 and validation metrics in Table 6 are intended as quick audit references for implementation and release reviews.

Table 5. Constants

Constant	Value
Structural shift threshold	`2.0 s`
Max absolute improvement cap	`25% of baseline`
Range base percentage	`1.5%`
Volatility contribution cap	`5%`
Missing-feature uncertainty multiplier	`2% * (1 - data_quality)`
ACWR instability range add	`+1%` outside `[0.6, 1.5]`
Dampening alpha bounds	`[0.3, 0.7]`
Riegel default exponent	`1.06`
Individual exponent bounds	`[1.01, 1.15]`
Readiness RF low/moderate boundary	`<0.35`, `<0.60`

Table 6. Accuracy and bias metrics

Metric	Equation
Error	`abs(actual - projected)`
Bias	`actual - projected`
MAE	`mean(errors)`
Bland-Altman lower	`bias_mean - 1.96 * std(biases)`
Bland-Altman upper	`bias_mean + 1.96 * std(biases)`

4. Verification of ML-Gap Closure Claims

The prior implementation thread reported closing major ML rollout gaps. Repository verification confirms:

1. Model orchestration seam exists and enforces rollout gating with deterministic fallback.
2. LSTM inference adapter exists, bounded outputs, and falls back when artifact/model is absent.
3. Lifecycle migration and persistence paths exist for model registry/jobs/studies/trials/artifacts.
4. Readiness injury-risk path exists with calibrated probability and persisted assessments.
5. Rollout observability endpoints exist (`/rollout/config`, `/rollout/metrics`, `/rollout/release-readiness`).

Open caveat: LSTM and Optuna paths remain scaffolded and rollout-gated; deterministic projection remains the default safety rail.

5. Reproducible PDF Build (Two-Column)

The PDF is generated from this markdown using a deterministic HTML+CSS build and Chrome headless print.

```
./docs/build_science_pdf.sh
```

This command emits:

- `docs/The_Science_Behind_PIRX.pdf` (final two-column paper)

6. References

- Riegel, P. S. (1981). Athletic records and human endurance.
- Blythe, D. A., and Kiraly, F. J. (2016). Predicting race times from training and race data.
- Lerebourg et al. (2023). KNN-based marathon prediction from runner profiles.
- PIRX implementation source of truth: repository modules listed in this document and root `README.md`.